MACHINE LEARNING AND BIO-INSPIRED OPTIMISATION

ASSIGNMENT 2

1. Introduction

We developed a Deep Reinforcement Learning agent to solve the Lunar Lander environment from OpenAI Gymnasium. The agent learns to land a spacecraft safely between two flags by interacting with its environment and improving its behaviour over time. We applied the Deep Q-Network (DQN) algorithm, a method that combines Q-Learning with deep neural networks to handle complex environments with large state spaces.

2. Problem Formulation

The Lunar Lander-v3 environment is a classic rocket trajectory optimisation problem with an 8-dimensional continuous state space and 4 discrete actions. The goal is to maximise the cumulative reward by:

- Landing softly between two flags.
- Avoiding crashes and minimising fuel usage.

3. Theory and Methodology

3.1 Deep Reinforcement Learning

Deep Reinforcement Learning (DRL) combines traditional reinforcement learning principles with the powerful function approximation abilities of deep neural networks. In reinforcement learning, an agent learns to interact with an environment by taking actions, observing rewards, and adjusting its behaviour to maximise cumulative rewards over time. However, in environments with large or continuous state spaces like lunar lander, it is impractical to store a table of all possible states and actions. This is where deep learning becomes essential: neural networks are used to estimate the action-value function (Q-function) directly from raw states.

3.2 Deep Q-Learning

Deep Q-Learning (DQN) is a specific Deep Reinforcement Learning method that extends Q-Learning by using a deep neural network to approximate the Q-function $Q(s,\ a)$. Rather than maintaining a Q-table, DQN learns a mapping from states to action-values.

Key components include:

- Experience Replay: Storing experiences and sampling randomly to train the network.
- Target Networks: Using a slowly updated network for stable training targets.
- ε-greedy Policy: A mechanism to balance between exploring new actions and exploiting learned actions.

3.3 Exploration and Exploitation (Problem 2)

In Deep Reinforcement Learning, the agent faces a constant trade-off between exploration and exploitation:

- Exploration means trying new actions to discover their effects, even if they might not immediately lead to the highest rewards.
- Exploitation means choosing the best-known action based on current knowledge to maximise the immediate reward.

In the Lunar Lander environment, exploration and exploitation play crucial roles:

- Early in training, the agent knows very little about how different actions affect the lander's movement and landing success. Therefore, exploration is essential to gather diverse experiences, such as firing different thrusters or adjusting the angle.
- As training progresses and the agent learns which actions lead to soft landings, it must exploit this knowledge by consistently selecting the best actions it has learned.

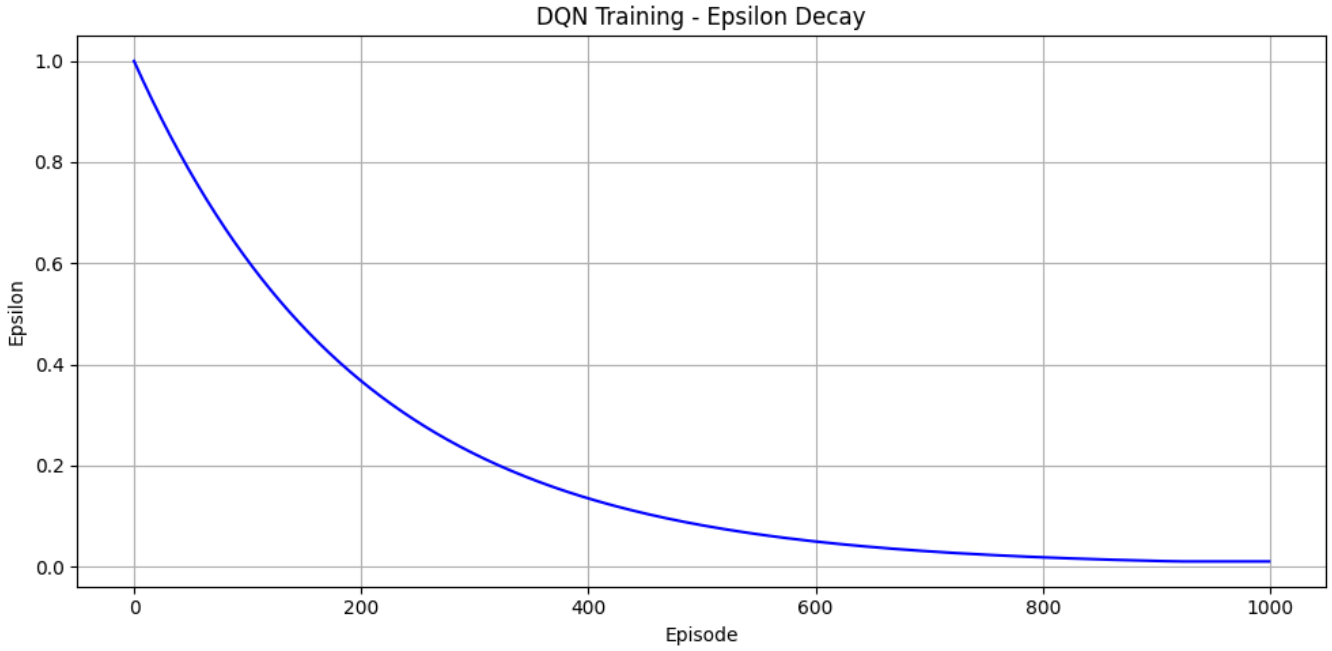

Figure 1: Epsilon value per episode

To manage this balance, we use an ε-greedy policy:

- With probability ε, the agent explores by selecting a random action.
- With probability 1 - ε, the agent exploits by choosing the action with the highest predicted reward from its Q-network.

Initially, ε is set high to encourage exploration. Over time, ε decays toward a smaller value, allowing the agent to exploit its learned strategies more often while still occasionally exploring to

avoid getting stuck in suboptimal behaviours. Figure 1 illustrates the decay of the epsilon (ε) value over the 1000 training episodes.

This balance ensures that the agent in Lunar Lander does not prematurely commit to poor strategies and can discover the best landing manoeuvres through continuous learning.

3.4 Action-Value Methods

Action-value methods focus on learning how good it is to take a certain action in a given state. In this case:

- The Q-network predicts the value of each action for the current state.
- The agent uses these values to choose actions.
- The agent updates its estimates according to the Q-learning formula:

$$Q(s, a) \leftarrow r + \gamma \cdot \max_{a'} Q(s', a')$$

$Q(s, a)$: Current estimate of the value of taking action a in state s.

r: Immediate reward received after taking action a.

γ: Discount factor that reduces the importance of future rewards.

s': New state reached after taking action a.

a': Possible next action from new state s'.

$\max_{a'} Q(s', a')$: Maximum predicted future value achievable from state s'.

Thus, the agent gradually learns which actions lead to higher long-term rewards.

4. Experimental Setup

4.1 Environment

We used the Lunar Lander-v3 environment from OpenAI Gymnasium. The default environmental settings were used.

Environment settings:

- continuous: False (discrete action space)
- gravity: -10.0 (gravity strength)
- enable_wind: False (no external wind during episodes)
- wind_power: 15.0 (strength of wind if enabled)
- turbulence_power: 1.5 (random turbulence effect)

Environment structure:

- State size: 8 (position, velocity, angle, leg contact indicators)
- Action size: 4 discrete actions (do nothing, fire left, fire main, fire right)

4.2 Deep Q-Learning Agent

Neural Network Architecture:

- **Input Layer**: 8 neurons (matching the state size).
- **Hidden Layers**: Two fully connected layers with ReLU activations.
- **Output Layer**: 4 neurons (one per action).

4.3 Hyperparameters

Hyperparameter	Value
Learning rate (lr)	0.001
Discount factor (γ)	0.99
Replay buffer size	100,000
Batch size	64
Soft update rate (τ)	0.001
Initial epsilon	1.0
Minimum epsilon	0.01
Epsilon decay rate	0.995
Optimiser	Adam

5. Methodology

- The agent is trained for 1000 episodes.
- The agent uses an ε-greedy strategy to balance exploration and exploitation.
- Save the experience (state, action, reward, next_state, done) in a replay buffer after each step.
- If the replay buffer has enough samples, a random batch of experiences is sampled and train the network.
- The target for training is computed using:

$$Q_{\text{target}} = \text{reward} + \gamma \times \max(Q_{\text{target\_network}}(\text{next\_state})) \times (1 - \text{done})$$

- We use Mean Squared Error (MSE) as the loss function and the Adam optimiser is used for updating the network parameters.

- The Q-network is updated by minimising the Mean Squared Error (MSE) between predicted and target Q-values.
- After every learning step, a soft update mechanism slowly updates the target network towards the local network to improve stability.

$$\theta_{\text{target}} \leftarrow \tau \times \theta_{\text{local}} + (1 - \tau) \times \theta_{\text{target}}$$

To monitor the agent's performance over 1000 episodes, we tracked:

- Total reward per episode
- Training loss per episode

We plotted these metrics to observe the learning trend.

6. Results

6.1 Reward Curve

The reward curve shows how the agent improves its landing skills over time. Initially, rewards are low (often crashing), but as training progresses, the rewards steadily increase. Successful landings generally correspond to rewards above 200. The reward trend becomes more stable toward the later episodes. Figure 2 shows the upward growth of the total episode rewards over time.

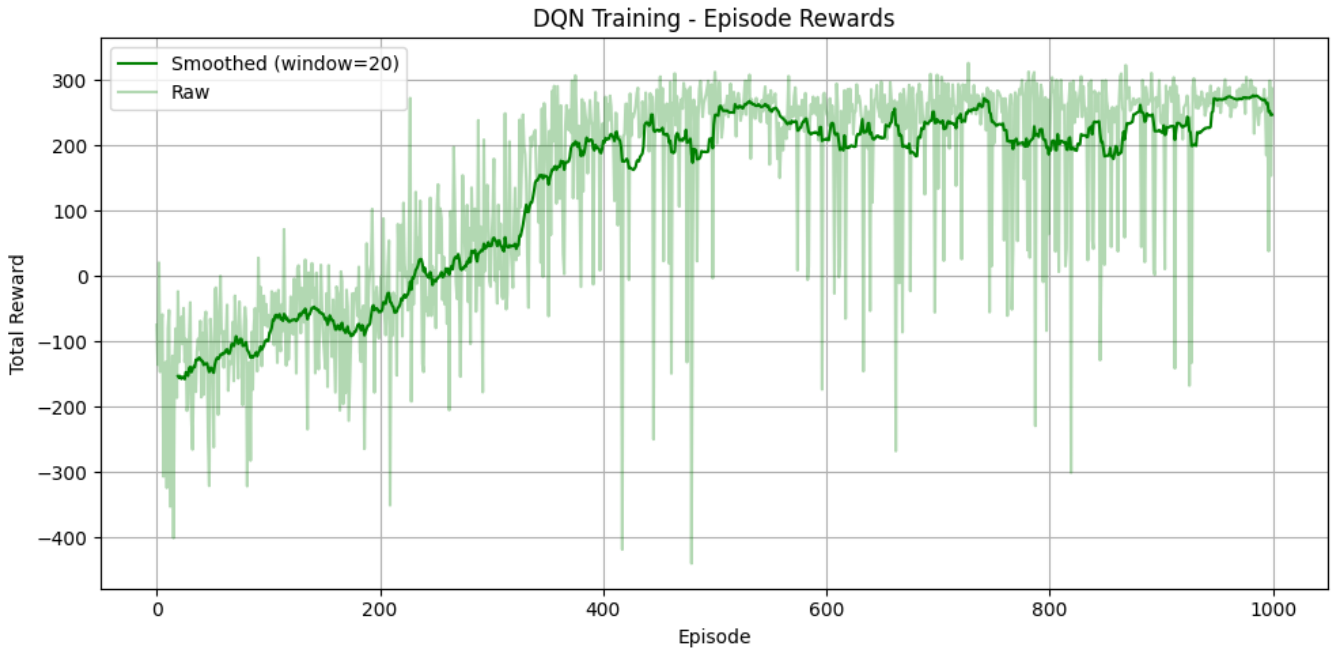

Figure 2: Total reward per episode with smoothing

6.2 Loss Curve

The loss curve shows the training error across episodes. Initially, the loss is high because the network is learning from random actions. As the agent gains experience, the loss decreases, indicating better predictions of action values. Figure 3 shows an initial sharp decrease in training loss and then a gradual flattening in the later episodes.

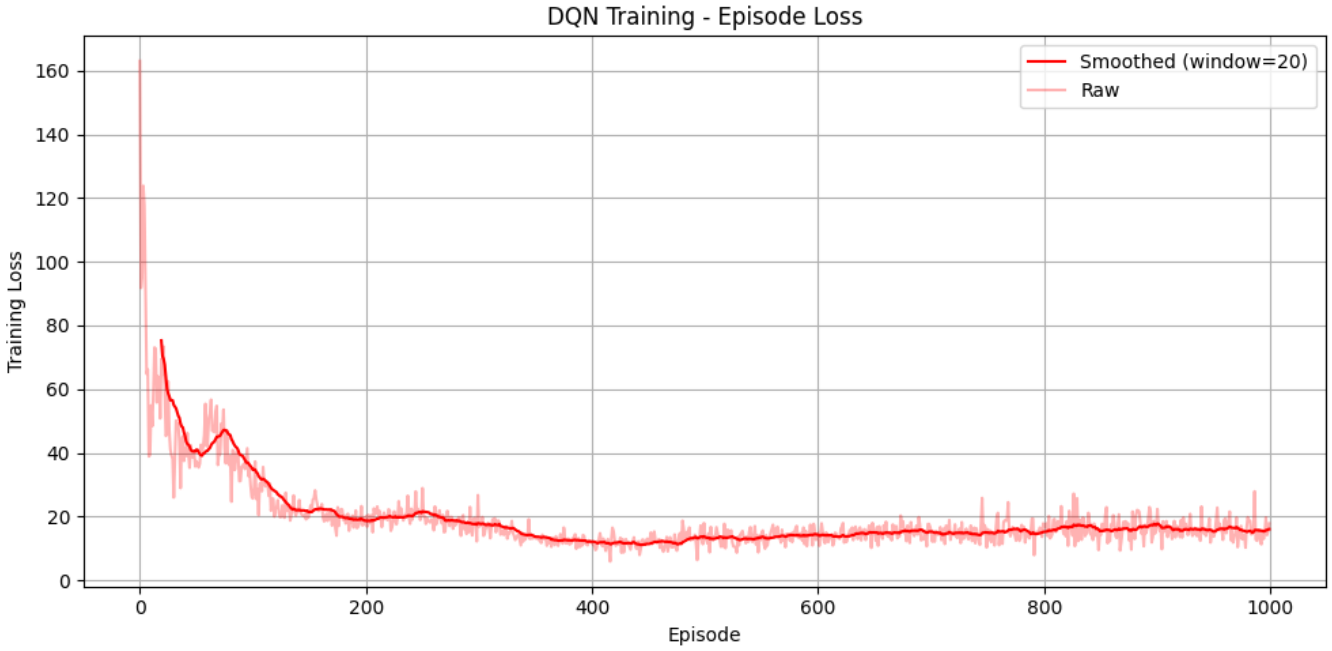

Figure 3: Total training loss per episode with smoothing

6.3 Demonstration of Agent Performance

To better illustrate how our agent interacts with the environment and learns over time, we include a video showcasing its behaviour.

6.4 Observations

- The agent successfully learned to land correctly after a few hundred episodes (after around 500 episodes).
- The cumulative reward consistently increased, showing that the policy was improving.
- The training loss showed a decreasing trend, confirming that the Q-value approximations were getting better.

By the end of training, the agent was able to land successfully in most episodes, achieving high rewards.

7. Conclusion

We built an agent that can master the Lunar Lander-v3 environment using Deep Q-Learning. The assignment demonstrated key reinforcement learning concepts such as:

- Function approximation via neural networks,
- Experience replays for stable learning,
- Target networks for improved convergence,
- ε-greedy policies for exploration.

References

[1] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA, USA: MIT Press, 2018. [Online]. Available: https://web.stanford.edu/class/psych209/Readings/SuttonBartoIPRLBook2ndEd.pdf

[2] Farama Foundation, "LunarLander — Gymnasium Documentation," [Online]. Available: https://gymnasium.farama.org/environments/box2d/lunar_lander/. [Accessed: Apr. 28, 2025].

Appendix

1. Contributions

Group Member	Contributions
Geo Raju 201855055 g.raju@liverpool.ac.uk	• Experiment design • DQN agent implementation • Report review and editing
Anurag Gupta 201833580 a.gupta8@liverpool.ac.uk	• NN implementation • Reward plotting • Report writing (Methodology & Results)
Anito Anto 201853825 a.anto3@liverpool.ac.uk	• DQN agent implementation • Refactored code • Report writing (Introduction, Methodology & Conclusions)
Nandana Puliyakkattil 201849311 n.puliyakkattil@liverpool.ac.uk	• NN implementation • GIF generation • Report writing (Theory)

Nancy 201845219 sgnancy@liverpool.ac.uk	• DQN agent implementation • Reward plotting • Report writing (Theory)
Prachi Kadam 201837271 p.kadam@liverpool.ac.uk	• DQN agent training • Environment setup • Report writing (Introduction, Experimental Setup)
Indudhar Chandrashekar 201824435 i.chandrashekar@liverpool.ac.uk	• Gym integration • Report writing (Introduction) • Formatting, and code review.

2. Video Link

https://docs.google.com/presentation/d/1IOZL72_z_jiqO1edNzhL_61cW8uvLzkZILKjQPxcljE/edit?usp=sharing